

PIXIU TECH LLP

Robotics Lab Book

Class 9 | Safety Systems, Displays and Autonomous Robots

Detect a fire before anyone smells smoke, put live data on a screen of your own, and build a robot that follows a painted path.

FIVE UNITS	Introduction Arduino IDE Basic Intermediate Final
FIFTEEN SESSIONS	One session = one class
THREE PROJECTS	Fire Alarm - LCD Weather System - Line Following Robot
BOARD	Arduino Uno (ATmega328P)
THIS BOOK BELONGS TO	
SCHOOL / SECTION	

Contents

Fifteen sessions. One session is one class period.

Unit 1 - Introduction	3
Session 1 - Systems That Sense, and What Is in Your Box	3
Session 2 - Electricity, Circuits and Signals	5
Check Yourself - Unit 1	9
Unit 2 - The Arduino IDE	9
Session 3 - Inside the Board and Inside the Chip	9
Session 4 - How Your Code Reaches the Chip	11
Session 5 - Reading the World: the ADC and Libraries	12
Check Yourself - Unit 2	15
Unit 3 - Basic Project: Fire Security Alarm System	15
Session 6 - Inside a Flame Sensor	15
Session 7 - Building the Alarm	16
Session 8 - Latching, Resetting and False Alarms	19
Check Yourself - Unit 3	22
Unit 4 - Intermediate Project: Smart 16x2 LCD Weather System	22
Session 9 - Inside a Liquid Crystal Display	22
Session 10 - Wiring and Driving the Display	24
Session 11 - Adding the Sensors and Rotating Screens	25
Check Yourself - Unit 4	29
Unit 5 - Final Project: Line Following Robot	29
Session 12 - Inside an IR Sensor, and Inside a Motor Driver	29
Session 13 - Building the Drive Base	31
Session 14 - The Four-Case Algorithm	33
Session 15 - Tuning, Track Testing and Your Demonstration	35
Check Yourself - Unit 5	37
Extra Challenges and Twenty Revision Questions	37
Words to Know	40
Answer Key	41
My Project Log	43

When you see	It means
Today you will	The three things this class is about. Read it before you start.
Behind the Scenes	How the component is actually built, and why it behaves as it does.
Try This	A short activity to do right now, at your desk, with your kit.
Watch Out	A mistake almost everyone makes. Read it before you wire.
Dark box	Arduino code. Type it exactly - every bracket and semicolon matters.
Challenge	For when you finish early and want something harder.

UNIT 1

Introduction

Sessions 1 and 2

Three machines wait for you this year: one that detects a fire, one that shows live data on its own screen, and one that drives itself along a line. All three rest on the same foundation, and this unit builds it.

By the end of this unit you will be able to:

- Explain the Sense - Think - Act idea with your own example.
- Name every component in your kit and say what it does.
- Build a circuit on a breadboard and explain why it works.
- Tell a digital signal from an analog one and say which pins handle each.

SESSION 1

one class

Systems That Sense, and What Is in Your Box

Today you will

- Understand what makes something a system rather than a gadget.
- Learn the names of all sixteen components in your kit.
- Agree the rules of the robotics lab.

A robot is any machine that can **sense** something about the world, **decide** what that means, and then **act**. The automatic door at a shop, the smoke alarm on your ceiling and the reversing camera in a car are all robots by that definition, though none of them looks like one.

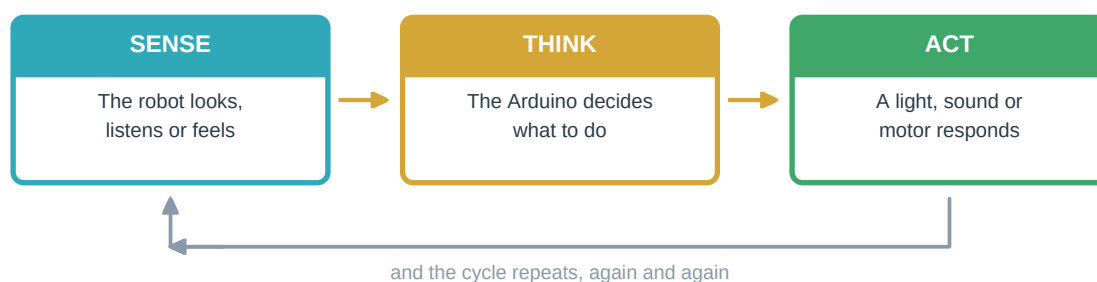


Figure 1.1 - Every robot in the world follows this same three-step loop

This year adds a word to that idea: **system**. A gadget does one thing. A system is several parts working together, where the behaviour of the whole is more than any single part. Your fire alarm will use two different sensors that must *agree* before it acts. Your robot will use two sensors whose readings change because of the robot's own movement. Thinking about the whole, not just the parts, is the real step up in Class 9.

Try This

Think about a lift. List everything it senses (buttons, floor position, door obstruction, weight), everything it decides, and everything it acts on. Then find one case where two of its senses must agree before it does anything. Every safety system in the world is built on that idea.

1.1 Open the Box

Lay everything out before you plug anything in. Learn the names first - it is impossible to ask for help with 'that small blue thing'. Your kit is grouped by unit, so the colour of each label tells you roughly when you will first need that part.

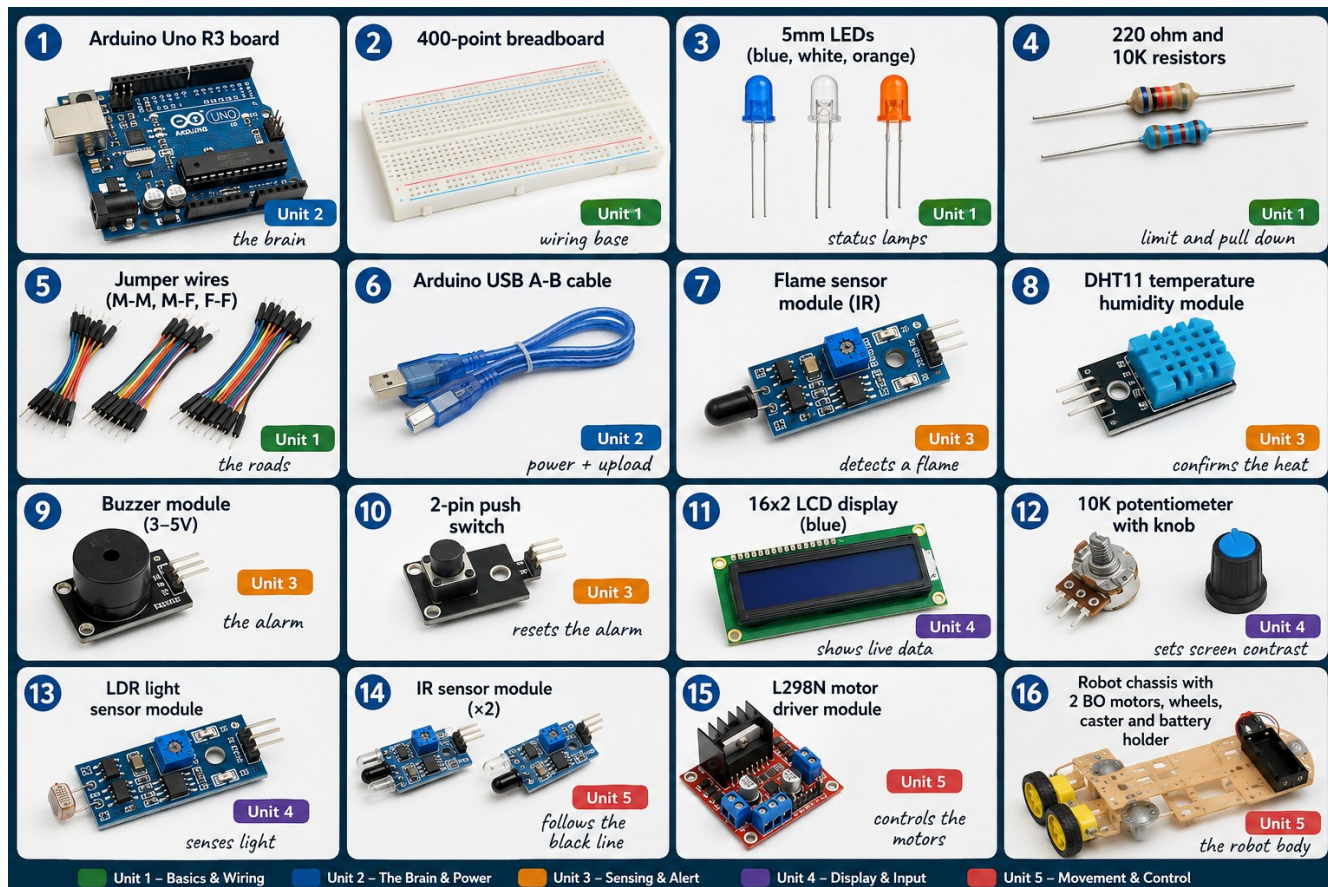


Figure 1.1 - Your kit. Learn these sixteen names before the next class.

Part	What it does	First used in
Arduino Uno board	The brain. It stores your program and runs it forever, even after the power has been off.	Session 3
USB A-B cable	Carries your program into the board and powers it while you test.	Session 3
400-point breadboard	Joins components together without any soldering.	Session 2
Jumper wires	The roads between parts. Male-to-male, male-to-female, female-to-female.	Session 2
5mm LEDs	Status lamps: normal, warning, alarm.	Session 2
220 ohm and 10K resistors	220 protects an LED; 10K pulls a switch input down to a known level.	Session 2
Flame sensor module	Detects the infrared light a fire gives off.	Session 6
DHT11 module	Measures temperature and humidity, to confirm what the flame sensor saw.	Session 7
Buzzer module	The alarm itself.	Session 7
2-pin push switch	Lets a human reset the alarm.	Session 8

Part	What it does	First used in
16x2 LCD display	Shows live readings with no laptop attached.	Session 9
10K potentiometer	Sets the contrast of the display.	Session 10
LDR light sensor module	Reports how bright the surroundings are.	Session 11
IR sensor modules (two)	Look down at the floor and tell black from white.	Session 12
L298N motor driver	A power amplifier for motors. The Arduino cannot drive them directly.	Session 12
Chassis with motors, wheels, caster and battery holder	The robot's body and its power supply.	Session 13

1.2 Rules of the Robotics Lab

- 1 Always unplug the USB cable before you change any wire.
- 2 Check the long leg of every LED and the + side of every module before power goes on.
- 3 Never run a wire straight from 5V to GND. That is a short circuit and it can kill the board.
- 4 **Never connect a motor directly to an Arduino pin.** It draws far more current than the pin can supply and the damage is permanent.
- 5 **Disconnect the battery pack before rewiring,** and never leave it connected at the end of a session.
- 6 **Flame testing only with a supervised candle in a metal tray,** away from paper, curtains and the battery pack.
- 7 Lift a robot off the floor before uploading, so it cannot drive away mid-upload.
- 8 Count every component back into the kit box before you leave.

Watch Out

Rule 6 is not a formality. You are building a fire detector, so at some point you must show it a real flame. One candle, one metal tray, one trainer watching, and nothing flammable within an arm's length. A fire alarm project that starts a fire is not a good demonstration.

SESSION 2

one class

Electricity, Circuits and Signals

Today you will

- Refresh how a circuit works and why it must be a loop.
- Light an LED on a breadboard with no code at all.
- Learn the difference between digital and analog signals.

Electricity behaves much like water in a pipe. The supply is the pump, the wires are the pipes, and each component is something the flow must pass through. **Voltage** is how hard the pump pushes, in volts. **Current** is how much flows each second, in amps. **Resistance** is anything that makes flowing harder, in ohms. These three are tied together by Ohm's law: $V = I \times R$.

That law is not decoration - it tells you why a 220 ohm resistor is the right partner for an LED. An LED drops about 2 volts across itself, leaving 3 volts across the resistor. Current is then 3 divided by 220, which is about 0.0136 amps, or 13.6 milliamps. That is comfortably inside both what the LED can survive and what an Arduino pin can supply. Choose 22 ohms instead and the current would be ten times higher, and something would fail.

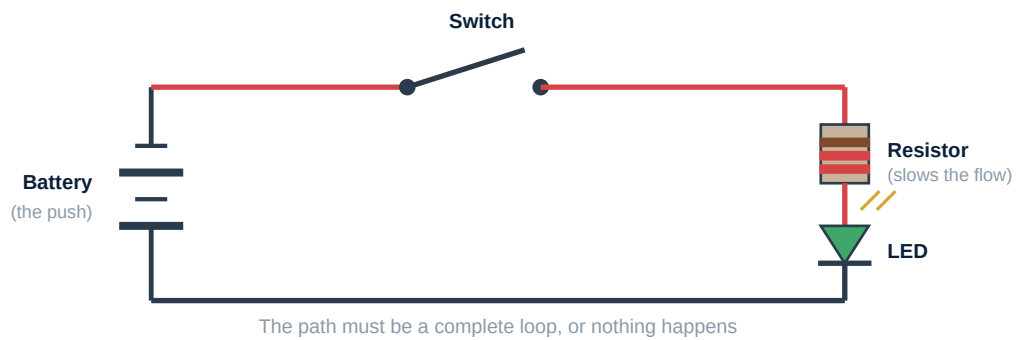
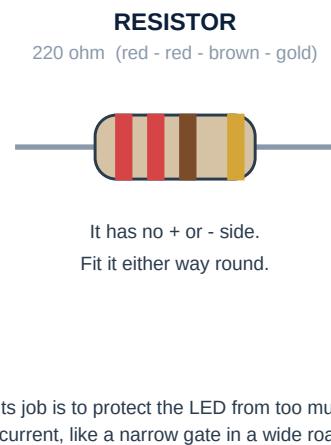
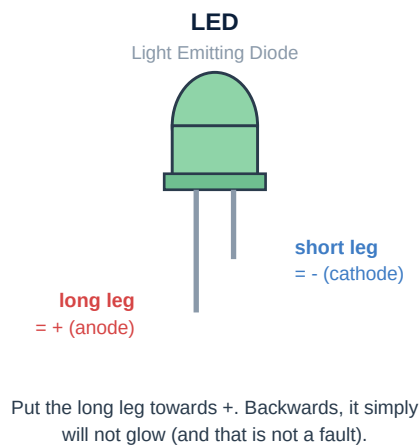


Figure 1.2 - A simple circuit: battery, switch, resistor and LED

**Watch Out**

An LED has a long leg and a short leg. Long leg towards +, short leg towards -. If an LED does not glow, turn it around before checking anything else. And it always needs a resistor: connect one straight to 5V and it will glow brilliantly for about a second, then never again.

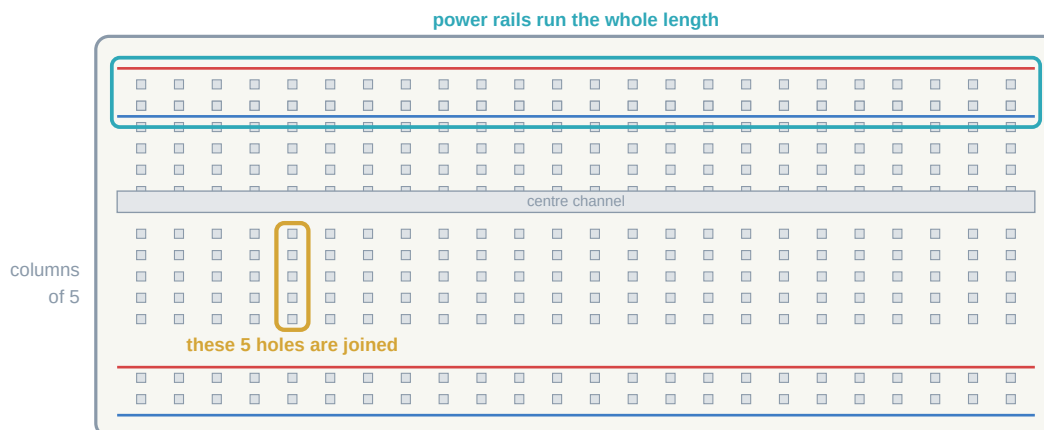
2.1 Inside the Breadboard

Figure 1.3 - Inside a breadboard: which holes are secretly connected

- The **long side rails** marked + and - run the whole length. Use them for power and ground.
- In the middle, holes are joined in **columns of five**, up and down only, never across.

- The **centre channel** splits the board in two. Nothing crosses it - which is exactly why chips are designed to straddle it.
- Two legs in the same column of five are connected; two legs in different columns are not.

Try This

Build the LED circuit with no code: LED legs in different columns, a 220 ohm resistor sharing the column with the long leg, resistor to 5V, short leg column to GND. Plug in the USB and it glows. Pull the GND wire and it dies. You have just proved the loop rule with your own hands.

2.2 Two Kinds of Signal

A push switch has two positions with nothing in between - a **digital** signal. A light level changes smoothly - an **analog** signal. The Arduino handles both: digital pins D0 to D13 deal in HIGH and LOW, while analog pins A0 to A5 measure a voltage and report a number from 0 to 1023.

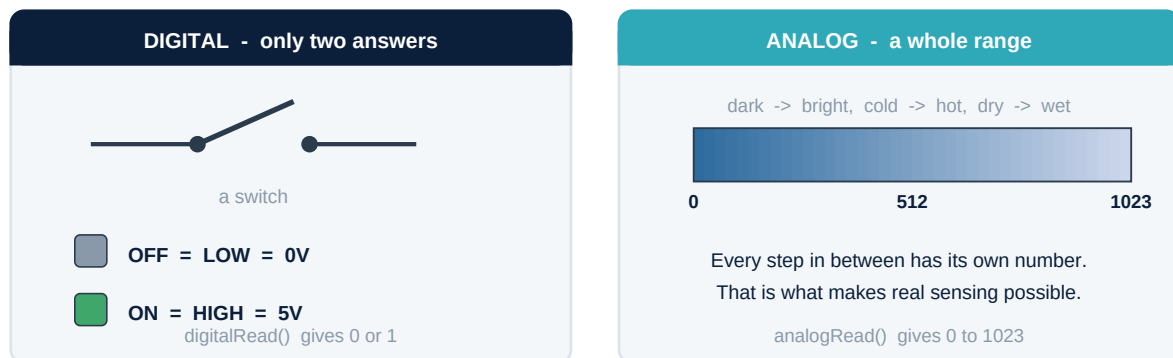


Figure 1.2 - The big idea of Class 7: numbers instead of yes-or-no

Your kit contains sensors of both kinds, and some that are cleverer than either. The LDR module gives you both: a raw analog value on AO and a simple yes-or-no on DO. The DHT11 is different again - it is a digital sensor that sends its readings as a coded sequence of pulses, which a library decodes for you. Knowing which kind you are holding decides which pin it goes to.

Sensor in your kit	Kind	Connect to
Push switch	Digital	any digital pin, with a 10K pull-down resistor
Flame sensor DO	Digital	any digital pin
Flame sensor AO	Analog	A0 to A5
LDR module AO	Analog	A0 to A5
Potentiometer wiper	Analog	A0 to A5
DHT11 DATA	Digital, coded	any digital pin, decoded by a library
IR sensor OUT	Digital	any digital pin

New word	What it means
Voltage	How hard electricity is pushed. Measured in volts.
Current	How much electricity flows each second. Measured in amps.
Ohm's law	$V = I \times R$. The relationship between voltage, current and resistance.
Pull-down resistor	A resistor that holds an input at 0V until a switch pulls it high.
System	Several parts working together, where the whole does more than any one part.

CHECK WHAT YOU LEARN'T

Check Yourself - Unit 1**A. Choose the correct answer**

- Ohm's law is written as
(a) $V = I + R$ (b) $V = I \times R$ (c) $V = I / R$ (d) $I = V \times R$
- A 220 ohm resistor with an LED lets through roughly
(a) 1.4 mA (b) 13.6 mA (c) 136 mA (d) 1.36 A
- In the middle of a breadboard, holes are joined in
(a) columns of five (b) rows of ten (c) no pattern (d) pairs
- The DHT11 sends its reading as
(a) a voltage (b) a coded sequence of pulses (c) a resistance (d) a colour
- A push switch needs a 10K resistor in order to
(a) limit current (b) hold the input at a known level (c) make it faster (d) protect the LED
- A system differs from a gadget because it
(a) is larger (b) has several parts working together (c) costs more (d) uses a battery

B. Fill in the blanks

- The push that moves electricity along is called _____.
- Analog pins on the Arduino Uno are labelled _____ to _____.
- A motor must never be connected directly to an _____ pin.
- Flame testing is done only with a supervised _____ in a metal tray.

C. Answer in one or two lines

- Name one safety system that requires two senses to agree before it acts.
- Work out the current through a 330 ohm resistor with 3 volts across it.
- Explain why the LDR module offers both an AO and a DO pin.
- Your LED does not glow. Write the first three things you would check.

UNIT 2

The Arduino IDE

Sessions 3, 4 and 5

Most people use an Arduino for years without ever asking what is actually happening inside it. This unit answers that question, because once you can picture the chip, the ADC and the upload process, debugging stops being guesswork.

By the end of this unit you will be able to:

- Name the main parts of the Arduino Uno board and say what each does.
- Describe what happens between pressing Upload and the program running.
- Explain what `analogRead()` actually measures and what its resolution is.
- Install and use an external library.

SESSION 3

one class

Inside the Board and Inside the Chip

Today you will

- Identify the main components on the Arduino Uno board.
- Learn what the ATmega328P contains.
- Upload Blink and confirm your whole setup works.

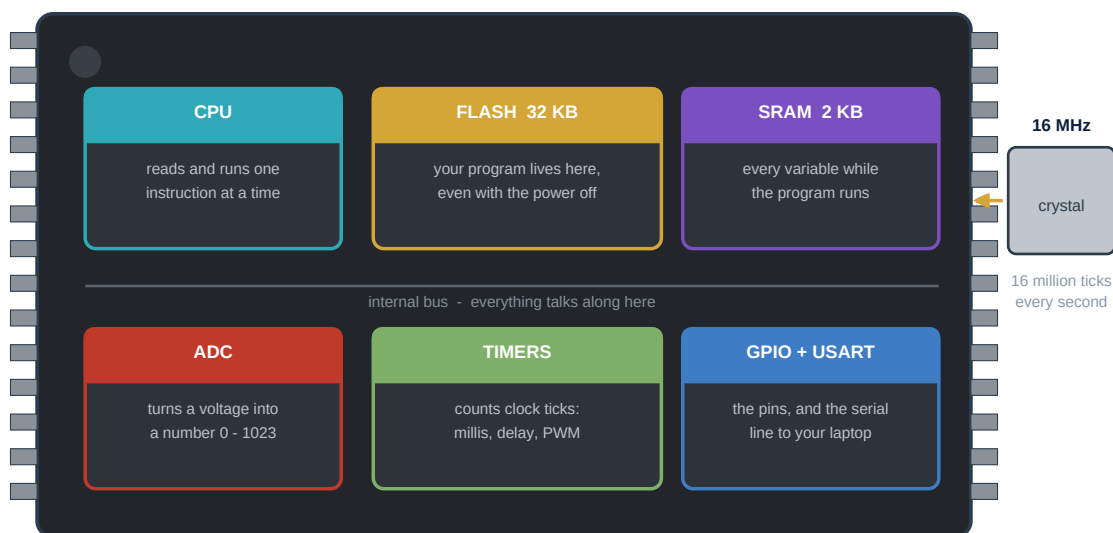
The blue board in your hand is not the computer. It is a **support system** for one chip. Look at it closely and you can find every part of that support system.

What you can see on the board	What it is for
The long black chip in the middle	The ATmega328P - the actual computer
A small silver oblong beside it	A 16 MHz quartz crystal that sets the chip's heartbeat
A second, smaller chip near the USB socket	Translates USB into the simple serial the ATmega understands
A three-legged part near the power jack	A voltage regulator, turning 7 to 12V into a steady 5V
The black barrel socket	Power in from a battery or adapter
Rows of black sockets	The headers - your pins D0 to D13 and A0 to A5
A small button	Reset - restarts your program from the beginning

Behind the Scenes

The ATmega328P costs a fraction of the board's price. Everything else on that PCB exists to make the chip easy for a human to use: to power it safely, to give it a steady clock, and to let a laptop talk to it over USB. A finished commercial product usually drops the board entirely and solders the bare chip onto its own circuit - which is exactly what happens inside a washing machine or a microwave.

ATmega328P - the chip inside the Arduino Uno



Everything you write ends up as instructions the CPU fetches from flash, one at a time, sixteen million times a second.

Figure 2.1 - Behind the blue board: what is actually doing the work

Inside the chip	Size	What it holds
Flash memory	32 KB	Your compiled program. Survives power loss.
SRAM	2 KB	Every variable while the program runs. Lost on power off.

Inside the chip	Size	What it holds
EEPROM	1 KB	Small amounts of data you want kept after power off.
CPU	-	Fetches and executes one instruction at a time.
ADC	10-bit	Turns a voltage into a number from 0 to 1023.
Timers	three	Count clock ticks. These run <code>millis()</code> , <code>delay()</code> and PWM.

Watch Out

Notice how small SRAM is: 2 KB for every variable at once. A single line like `Serial.println("Temperature is now");` stores that whole sentence in SRAM. Programs with many long messages can quietly run out of memory and behave very strangely - and the compiler will not always warn you.

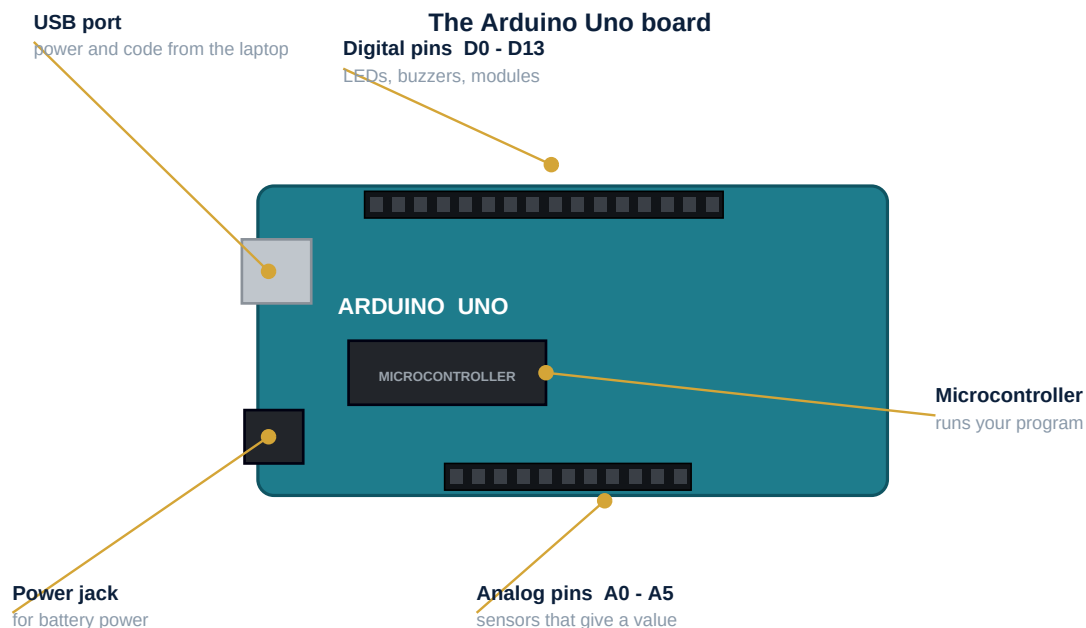


Figure 2.1 - Know your board before you wire anything to it

SESSION 4

one class

How Your Code Reaches the Chip**Today you will**

- Follow your program from typed text to running machine code.
- Learn what the bootloader does and why the board resets.
- Read an error message and work out what it means.

When you press Upload, four separate things happen in about two seconds. Most people never learn them, and then cannot explain why an upload failed. You will be able to.

- 1 Compiling.** Your English-like text is translated into machine instructions the ATmega understands. Any spelling or grammar mistake stops the process here, and nothing is sent.
- 2 Linking.** The libraries you included are joined to your code, and the whole thing becomes one file of hexadecimal numbers - the `.hex` file.
- 3 Resetting.** The IDE briefly pulls the board's reset line. The chip restarts and, for about a second, runs a small program already built into it called the **bootloader**.

- 4 **Writing.** The bootloader listens on the serial line, receives your .hex file, and writes it into flash memory. Then it hands control to your program, which starts running immediately.

Behind the Scenes

That one-second bootloader window explains a fault you will certainly meet. If your program starts talking on the serial line the instant it boots, it can shout over the bootloader and the next upload fails. It also explains why the two small LEDs near the USB socket flicker during upload - that is your program travelling down the wire, one packet at a time.

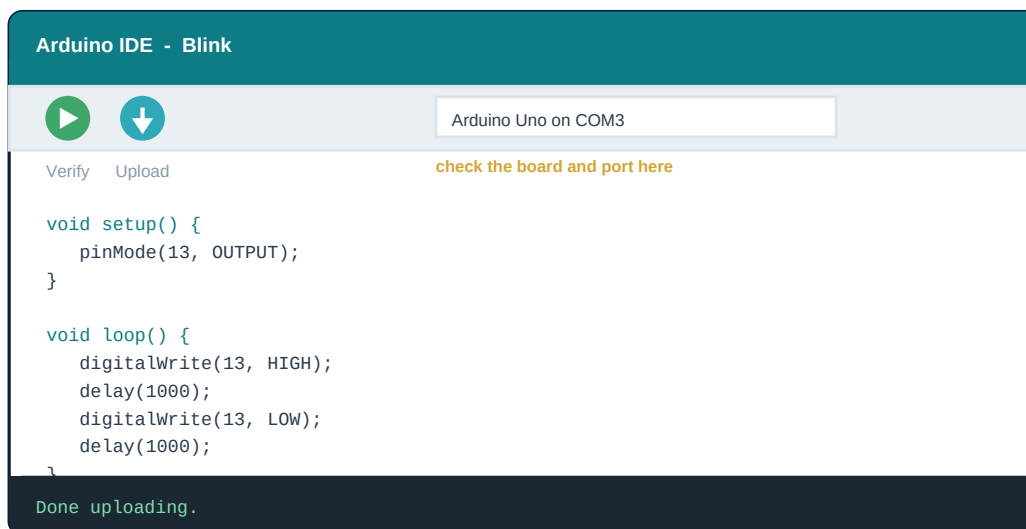


Figure 2.2 - Verify checks your code; Upload sends it to the board

What the screen says	Which stage failed	How to fix it
expected ';' before ...	Compiling	Add a semicolon at the end of the line above
'digitalwrite' was not declared	Compiling	Capitals matter: digitalWrite
no matching function for call to ...	Compiling	Wrong number or type of values passed
Board at COM__ is not available	Resetting	Re-select Tools > Port, or change cable
programmer is not responding	Writing	Wrong board selected, or the bootloader was interrupted
Sketch too big	Linking	Your program exceeds 32 KB of flash
Low memory available	Linking	Too many variables or long strings for 2 KB of SRAM

Blink.ino - the test that proves the whole chain works

```
int led = 13;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  digitalWrite(led, HIGH);
  delay(1000);
  digitalWrite(led, LOW);
  delay(1000);
  Serial.println(millis()); // how long the board has been running
}
```

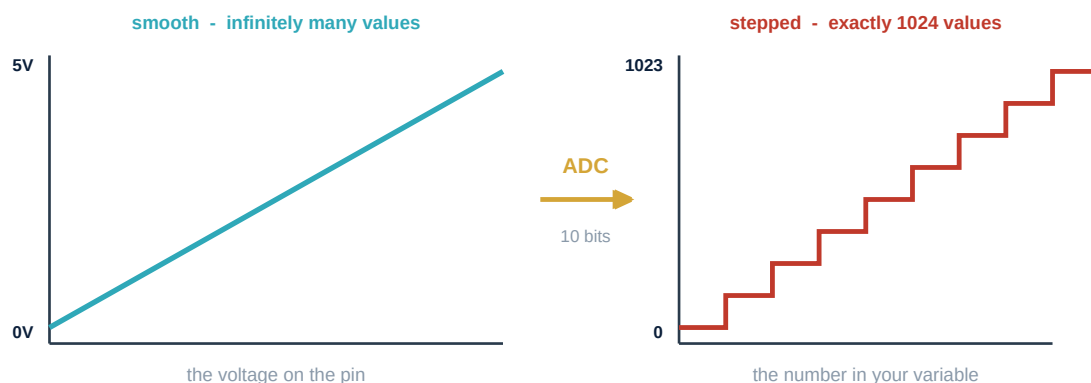
Reading the World: the ADC and Libraries

Today you will

- Understand what `analogRead()` actually does inside the chip.
- Work out the resolution of a 10-bit ADC.
- Install a library and use it.

An analog pin does not measure light, or heat, or distance. It measures exactly one thing: **voltage**. Inside the ATmega there is a circuit called an **Analog to Digital Converter**, and its job is to compare the voltage on the pin against a known reference and report the answer as a whole number.

Analog to Digital Conversion



$$\text{one step} = 5V / 1024 = \text{about } 4.9 \text{ millivolts}$$

a change smaller than that is invisible to the Arduino. That is its resolution.

Figure 2.2 - Every `analogRead()` is a measurement rounded to the nearest step

The ATmega's ADC is **10-bit**, which means it reports 2 to the power 10 different values - 1024 of them, numbered 0 to 1023. Spread across a 5 volt range, each step is 5 divided by 1024, or about 4.9 millivolts. That number is the sensor's **resolution**: any change smaller than one step is simply invisible.

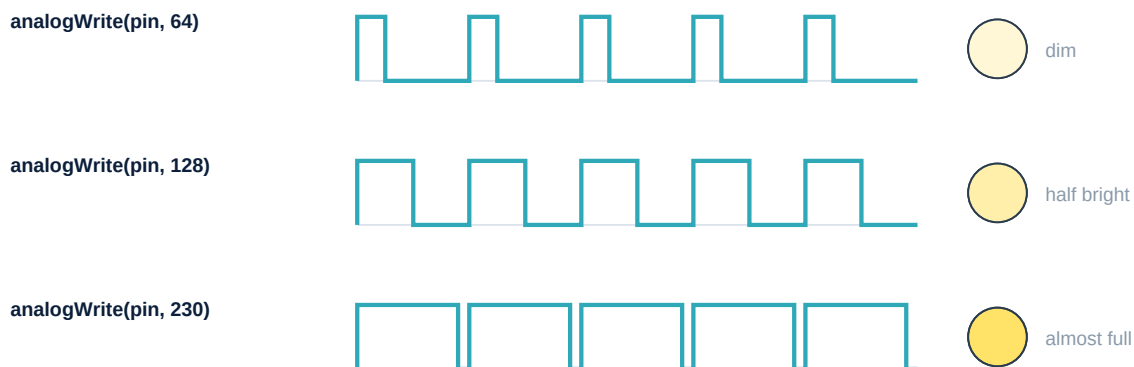
Voltage on the pin	What <code>analogRead()</code> gives	Working
0 V	0	nothing to measure
1.25 V	about 256	$1.25 / 5 \times 1023$
2.5 V	about 512	half of full scale
5 V	1023	full scale

Behind the Scenes

The ADC does not measure instantly. It uses a technique called successive approximation: it guesses half, asks *is the voltage above this?*, then halves the remaining range and asks again, ten times over. Each `analogRead()` therefore takes about 100 microseconds. That is fast, but it is not free - and in a tight loop reading six sensors, it adds up.

5.1 The Opposite Trick: PWM

If the ADC turns a voltage into a number, PWM does something close to the reverse. A digital pin cannot produce half of 5 volts - it is only ever fully on or fully off. So the chip cheats: a timer switches the pin on and off about 490 times a second, and by changing how much of each cycle is spent on, the *average* voltage changes. Your eye, and a motor's inertia, are far too slow to notice the switching.

PWM - switching so fast that your eye sees an average

The pin is still only ON or OFF. It is the amount of time spent ON that changes.

Only pins marked with ~ can do this: D3, D5, D6, D9, D10 and D11.

Only six pins on the Uno are wired to those timers - D3, D5, D6, D9, D10 and D11 - and they are marked with a small ~. You will use PWM this year to set the speed of your robot's motors, where `analogWrite(ena, 130)` means *keep the enable pin on for about half of each cycle*.

5.2 Libraries: Standing on Someone Else's Work

Some components are far too complicated to talk to by hand. The DHT11 sends its reading as a precisely timed sequence of forty pulses. The 16x2 LCD needs a specific start-up ritual before it will show anything at all. Decoding these yourself would take hundreds of lines.

A **library** is a bundle of instructions somebody else has already written and tested. You include it once, and then a single sensible line does the work. This is not cheating - it is how all professional software is built.

- 1 Open **Tools > Manage Libraries** in the IDE.
- 2 Search for **DHT sensor library** and install the one by **Adafruit**.
- 3 If it offers **Adafruit Unified Sensor** as well, accept - it depends on it.
- 4 The LiquidCrystal library for your display is already built in; nothing to install.
- 5 Close the manager. The new instructions are now available to every sketch.

New word	What it means
ATmega328P	The microcontroller chip that is the real computer on the board.
Flash / SRAM	Memory for the program / memory for variables while running.
Bootloader	A small built-in program that receives your code and writes it to flash.
ADC	The circuit that turns a voltage into a number.
Resolution	The smallest change a measuring system can detect.
Library	Ready-made code you include and use.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 2**A. Choose the correct answer**

1. The actual computer on an Arduino Uno is
(a) the USB socket (b) the ATmega328P chip (c) the blue board (d) the crystal
2. The 16 MHz crystal provides
(a) power (b) the clock (c) memory (d) the reset
3. Your compiled program is stored in
(a) SRAM (b) flash memory (c) EEPROM (d) the ADC
4. The ATmega's ADC is 10-bit, so it reports
(a) 10 values (b) 256 values (c) 1024 values (d) 1023 volts
5. One ADC step across a 5V range is about
(a) 4.9 millivolts (b) 49 millivolts (c) 0.5 volts (d) 5 volts
6. The bootloader's job is to
(a) run your program (b) receive your code and write it to flash (c) measure voltage (d) power the board

B. Fill in the blanks

- 1 Variables live in _____ while the program is running.
- 2 A 10-bit ADC reports numbers from _____ to _____.
- 3 The smallest change a system can detect is called its _____.
- 4 Ready-made code written by somebody else is called a _____.

C. Answer in one or two lines

- 1 Describe the four stages between pressing Upload and your program running.
- 2 A pin sits at 3.75 volts. Roughly what will `analogRead()` return? Show your working.
- 3 Explain why a program with many long text messages may run out of memory.
- 4 Why is using a library not cheating?

UNIT 3

Basic Project: Fire Security Alarm System

Sessions 6, 7 and 8

Your first machine has a serious job. It must notice a fire quickly, be certain enough not to cry wolf, and once it has decided, refuse to go quiet until a human says so. Every one of those requirements changes the code.

By the end of this unit you will be able to:

- Explain how a flame sensor detects fire without measuring heat.
- Combine two sensors so that both must agree before an alarm sounds.
- Implement a latching alarm with a manual reset.
- Test honestly for false positives and false negatives.

SESSION 6

one class

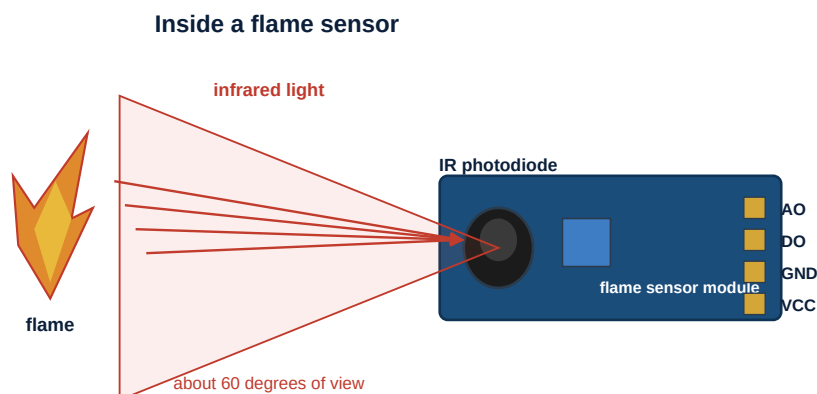
Inside a Flame Sensor

Today you will

- Learn what a flame sensor actually detects.
- Understand why a second sensor is needed to confirm.
- Take your first readings from a supervised candle.

The obvious way to detect a fire is to measure temperature. It is also the wrong way. Air is a poor conductor of heat, so a thermometer a metre from a flame may take minutes to notice anything - and in a fire, minutes are everything.

A flame sensor takes a completely different approach. It does not measure heat at all. It watches for **light** - specifically infrared light in the band from about 760 to 1100 nanometres, which is exactly the band a burning flame emits strongly. Light travels at 300,000 kilometres per second, so the sensor reacts in milliseconds.



Why it is not a heat sensor

A flame gives off infrared light at around 760 to 1100 nanometres. The black dome is a photodiode tuned to exactly that band, so it is nearly blind to ordinary room light. It therefore reacts in milliseconds - long before the air around it has warmed up at all. But sunlight and filament bulbs also emit infrared, which is why a second sensor is used to confirm before an alarm is raised.

Figure 3.1 - It sees the light of a fire, not the heat of one

Behind the Scenes

The black dome is a **photodiode**, and the black plastic is not decoration - it is an optical filter. It blocks visible light while letting infrared through, which is why the sensor barely reacts to a torch shone straight at it. The same black plastic is used on your TV remote's receiver, for exactly the same reason.

But there is a catch, and it is the reason this project needs two sensors. Sunlight is full of infrared. So is a filament bulb, and so is a hot soldering iron. A flame sensor on its own will happily announce a fire because somebody opened a curtain. The fix is **confirmation**: require a second, independent measurement to agree before acting.

Sensor	What it sees	How fast	Foiled by
Flame sensor	Infrared light from a flame	milliseconds	sunlight, filament bulbs, hot metal
DHT11	Air temperature rising	tens of seconds	a warm day, a heater, your hand
Both together	IR light AND rising temperature	seconds	almost nothing in a classroom

Try This

Wire the flame sensor: VCC to 5V, GND to GND, AO to A0, DO to D2. Print the AO value on the Serial Monitor. Note the number in a normal room, then near a window, then with a torch pointed at it, and finally with a supervised candle at 30 cm. Those four numbers tell you where your threshold must sit.

SESSION 7

one class

Building the Alarm

Today you will

- Wire both sensors, the buzzer, the lamps and the reset switch.
- Understand why a switch needs a pull-down resistor.
- Get a first working alarm running.

Before wiring, it is worth knowing what your confirming sensor actually contains. The DHT11 is two instruments and a small computer in one blue box.

Inside the DHT11	How it works	What limits it
Humidity sensor	Two plates with a moisture-absorbing film between them. As the film takes in water, its ability to store charge changes, and the chip measures that change.	Only whole percentages, plus or minus 5
Temperature sensor	A thermistor - a resistor whose resistance falls sharply as it warms. The chip compares it against a fixed reference.	Whole degrees, plus or minus 2
Small controller	Converts both readings and sends all forty bits as one precisely timed train of pulses down the single DATA wire.	One reading every two seconds

Behind the Scenes

That single DATA wire is doing something remarkable. The Arduino pulls it low to ask for a reading; the DHT11 replies with forty pulses where a short pulse means 0 and a long one means 1. Getting the timing right is fiddly to the microsecond, which is exactly why a library exists. It also explains the two-second rule: the sensor genuinely needs that long to take a fresh measurement.

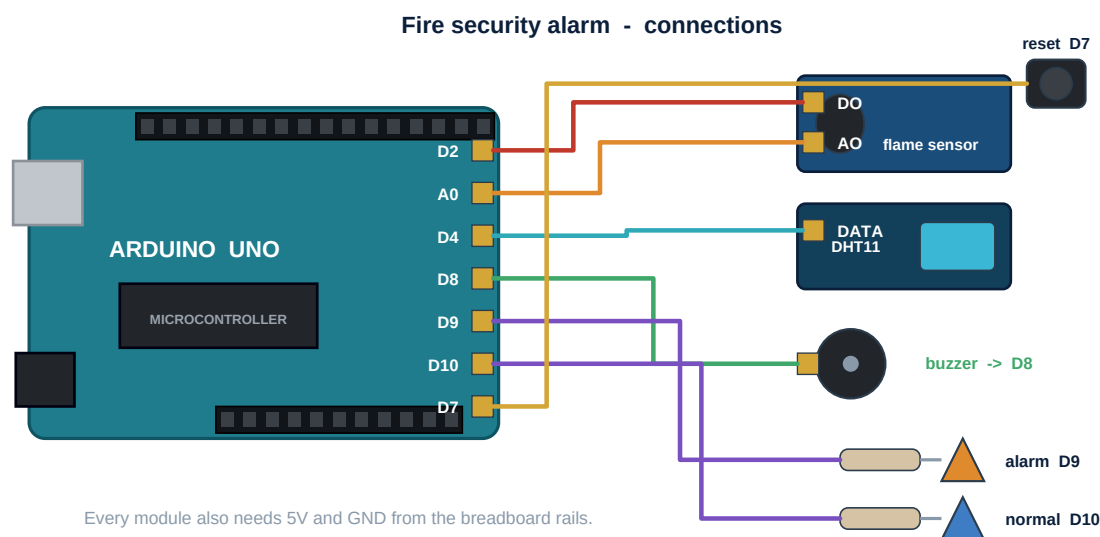


Figure 3.2 - Two sensors in, three alerts out, one reset

Component pin	Arduino pin	Why
Flame sensor DO	D2	A quick yes-or-no trip
Flame sensor AO	A0	How strong the infrared is
DHT11 DATA	D4	Temperature, for confirmation
Buzzer +	D8	The alarm
Orange LED long leg	D9 through 220 ohm	Alarm lamp
Blue LED long leg	D10 through 220 ohm	Normal lamp

Component pin	Arduino pin	Why
Push switch, one side	5V	Supply side
Push switch, other side	D7, and 10K to GND	Reset input with a pull-down

Behind the Scenes

Why does the switch need that 10K resistor? Because an unconnected input pin is not at 0V - it is **floating**, picking up stray electrical noise from the air and the wires, and reading HIGH and LOW at random. The 10K resistor gently ties the pin to ground so it reads a firm LOW, while still being weak enough that pressing the switch easily overpowers it and pulls the pin to 5V. Ten thousand ohms is the standard compromise.

7.1 Behind the Scenes: How a Buzzer Makes Sound

Prise the sticker off a buzzer and you would find a thin brass disc with a layer of ceramic bonded to it. That ceramic is **piezoelectric**: apply a voltage and it physically changes shape, bending the disc a fraction of a millimetre. Remove the voltage and it springs back.

Do that thousands of times a second and the disc pushes the air thousands of times a second - and that is sound. Your kit contains an **active** buzzer, which has its own oscillator built in, so a plain `digitalWrite(buzzer, HIGH)` produces a note. A **passive** buzzer has no oscillator, and you would have to supply the frequency yourself with `tone(pin, 2000)`.

	Active buzzer	Passive buzzer
How you drive it	<code>digitalWrite HIGH</code>	<code>tone(pin, frequency)</code>
Pitch	fixed by the maker	any pitch you choose
Best for	alarms and alerts	melodies and warning patterns

7.2 Behind the Scenes: Why a Switch Bounces

A push switch feels like a clean action to your finger, but electrically it is not. The two metal contacts are springy, and when they meet they physically bounce apart and back together several times in the first few milliseconds. To a chip running sixteen million operations a second, one press can therefore look like five or six separate presses.

The cure is called **debouncing**. The simplest version is the one in your alarm code: after detecting a press, wait a few hundred milliseconds before looking again, by which time the chatter has finished. It is crude but entirely effective for a reset button.

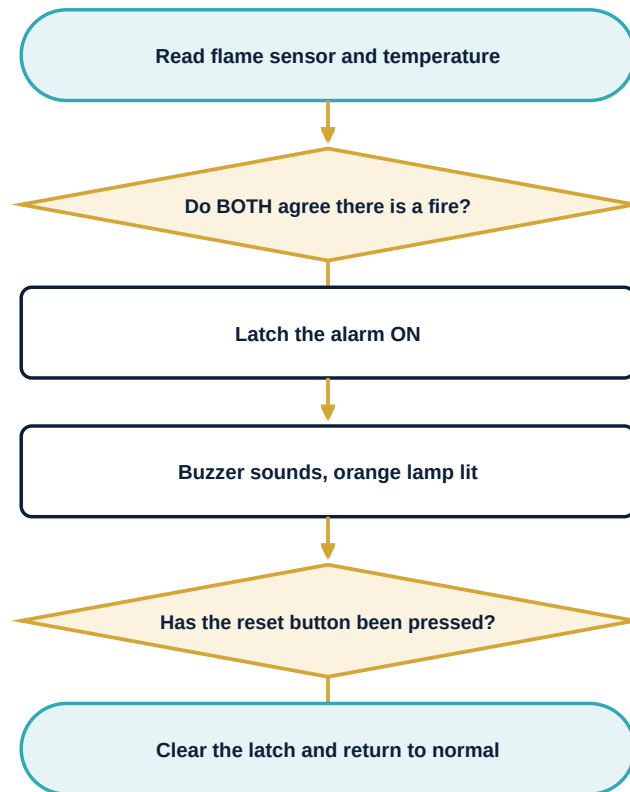


Figure 3.3 - Confirmation, then latching, then a human reset

SESSION 8

one class

Latching, Resetting and False Alarms

Today you will

- Understand why an alarm must latch.
- Write the full program with confirmation and reset.
- Test for false positives and false negatives.

Here is a requirement that changes everything. Suppose a flame flares for half a second and then goes out. A simple if statement would sound the buzzer for that half second and then fall silent, and nobody would ever know. A real alarm must **latch**: once triggered, it stays on until a person deliberately clears it.

Latching is done with a **boolean** - a variable that holds only true or false. The sensors can set it to true, but only the reset button can set it back to false. That single idea turns a reaction into a memory.

FireSecurityAlarm.ino

```

#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

int flameDO = 2, flameAO = A0;
int buzzer = 8, alarmLed = 9, normalLed = 10, resetBtn = 7;

int flameLimit = 300; // below this on A0 means strong infrared
float tempLimit = 38.0; // degrees Celsius - set from your own data
  
```

```

boolean alarmLatched = false; // this is the machine's memory
int alarmCount = 0;

void setup() {
  pinMode(flameDO, INPUT);
  pinMode(buzzer, OUTPUT);
  pinMode(alarmLed, OUTPUT);
  pinMode(normalLed, OUTPUT);
  pinMode(resetBtn, INPUT);
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  int ir = analogRead(flameAO);
  float t = dht.readTemperature();

  if (isnan(t)) { t = 0; } // sensor did not answer this time

  boolean seesFlame = (ir < flameLimit);
  boolean isHot = (t > templimit);

  Serial.print("IR: "); Serial.print(ir);
  Serial.print(" T: "); Serial.print(t);
  Serial.print(" latched: "); Serial.println(alarmLatched);

  if (seesFlame && isHot) { // BOTH must agree
    if (!alarmLatched) { // only count a new alarm once
      alarmCount = alarmCount + 1;
    }
    alarmLatched = true;
  }

  if (digitalRead(resetBtn) == HIGH) {
    alarmLatched = false; // only a human can clear it
    delay(300); // simple debounce
  }

  if (alarmLatched) {
    digitalWrite(alarmLed, HIGH);
    digitalWrite(normalLed, LOW);
    digitalWrite(buzzer, HIGH);
    delay(150);
    digitalWrite(buzzer, LOW);
    delay(150);
  } else {
    digitalWrite(alarmLed, LOW);
    digitalWrite(normalLed, HIGH);
    digitalWrite(buzzer, LOW);
    delay(300);
  }
}

```

Read the two lines that create `seesFlame` and `isHot`. A comparison such as `ir < flameLimit` is already true or false, so it can be stored straight into a boolean. Naming those two conditions makes the important line - `if (seesFlame && isHot)` - read almost like English. Code that reads like its own explanation is code you can still understand next month.

8.1 Testing Honestly

Every detector in the world can be wrong in two different ways, and engineers name them separately because the consequences are so different.

Name	What happens	Why it matters
False positive	Alarm sounds when there is no fire	People stop trusting the alarm and eventually disconnect it
False negative	There is a fire and the alarm stays silent	The alarm has failed at the only job it had

Test	Should the alarm fire?	Did it?	Tick
Normal classroom, nothing happening	No	_____	■
Bright torch shone at the sensor	No	_____	■
Sunlight through an open window	No	_____	■
Warm hand cupped over the DHT11	No	_____	■
Supervised candle at 30 cm	Yes, within 2 seconds	_____	■
Candle removed	Stays latched	_____	■
Reset button pressed	Returns to normal	_____	■

Challenge

Add a second stage to your alarm. If the flame sensor alone trips, show a flashing orange lamp as a *warning* but stay silent. Only when the temperature confirms should the buzzer sound. Real industrial detectors work exactly this way, and it is called a two-stage alarm.

New word	What it means
Photodiode	A component that produces a current when light falls on it.
Infrared	Light just beyond red, invisible to us, given off strongly by flames.
Latching	Staying on once triggered, until deliberately cleared.
Boolean	A variable that holds only true or false.
Floating input	An unconnected pin, reading randomly because of stray noise.
False positive / negative	Alarming with no fire / staying silent during one.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 3**A. Choose the correct answer**

1. A flame sensor detects a fire by sensing
(a) heat (b) smoke (c) infrared light (d) sound
2. The black dome on the sensor acts as
(a) a lens (b) an optical filter (c) a heat sink (d) decoration
3. Two sensors are used together in order to
(a) measure faster (b) reduce false alarms (c) save power (d) look impressive
4. A latching alarm stays on until
(a) the fire goes out (b) a human resets it (c) the power dips (d) 10 seconds pass
5. A pin with nothing connected to it is described as
(a) grounded (b) floating (c) latched (d) powered
6. An alarm that stays silent during a real fire is a
(a) false positive (b) false negative (c) latch (d) debounce

B. Fill in the blanks

- 1 A flame emits infrared strongly between about _____ and _____ nanometres.
- 2 A variable holding only true or false is called a _____.
- 3 The switch input needs a _____ resistor to stop it floating.
- 4 An alarm that sounds with no fire present is a false _____.

C. Answer in one or two lines

- 1 Explain why measuring temperature alone is a poor way to detect fire quickly.
- 2 Write the IR and temperature values you measured, and the limits you chose.
- 3 Why is a false positive dangerous even though nobody is hurt by it?
- 4 Explain in your own words what latching does and why it is needed.

UNIT 4

Intermediate Project: Smart 16x2 LCD Weather System

Sessions 9, 10 and 11

Until now, seeing your data has meant staring at a laptop. This unit cuts the cord. Your machine gets a screen of its own, and with it the ability to stand alone on a shelf and be useful to somebody who has never seen your code.

By the end of this unit you will be able to:

- Explain how a liquid crystal display forms a character.
- Wire a parallel display in 4-bit mode and explain each control line.
- Present multi-sensor data in a readable, human-facing format.
- Track minimum and maximum values over time.

SESSION 9

one class

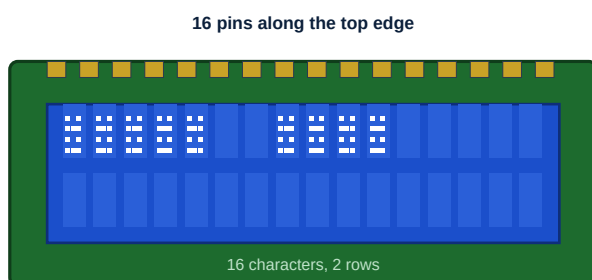
Inside a Liquid Crystal Display

Today you will

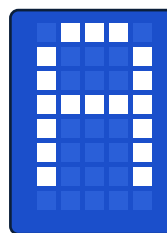
- Learn how liquid crystal actually works.
- Understand what the HD44780 controller does for you.
- Identify all sixteen pins on the display.

The name sounds like a contradiction. How can something be both liquid and crystal? The answer is that liquid crystal is a genuine fourth state of matter. Its molecules flow freely like a liquid, but they also line up with one another like a crystal - and crucially, that alignment can be changed by a small voltage.

Inside a 16x2 character LCD



a 5 x 8 grid of dots per character



the HD44780 controller stores the dot pattern for 208 characters

Why it is called liquid crystal

Between two sheets of glass sits a liquid whose long molecules normally line up in a twist. That twist rotates light passing through, so it escapes the far filter and the pixel looks pale. Apply a small voltage and the molecules straighten. The light no longer twists, the far filter blocks it, and that dot goes dark. No dot makes its own light - the backlight behind does that, which is why an LCD needs its LED backlight and why the contrast pot changes what you can read.

Behind the Scenes

Notice what this means: an LCD dot produces **no light of its own**. It only blocks or passes the light of the backlight behind it. That is why the display is unreadable if the backlight fails, why the contrast potentiometer matters so much, and why an LCD uses so little power compared with a screen that generates its own light. It is also why a calculator screen still works in bright sun and a phone screen struggles.

Driving thousands of individual dots would be impossible from an Arduino. Fortunately the black blob on the back of your display is a dedicated controller chip, the **HD44780**. It has been the industry standard since the 1980s and it does a great deal for you: it stores the dot pattern for 208 characters, keeps track of where the cursor is, and refreshes every dot continuously. You simply send it a letter, and it draws it.

Pin	Name	What it is for
1, 2	VSS, VDD	Ground and 5V power
3	V0	Contrast. This is what the potentiometer sets.
4	RS	Register Select: is this byte a character, or a command?
5	RW	Read or Write. Tied to ground - we only ever write.
6	E	Enable. A pulse here means 'take the data now'.
7 to 14	D0 to D7	The data lines. In 4-bit mode we use only D4 to D7.

Pin	Name	What it is for
15, 16	A, K	The LED backlight. A goes to 5V through a 220 ohm resistor.

Behind the Scenes

Why use only four of the eight data lines? Because eight would eat eight Arduino pins and leave almost nothing for your sensors. In **4-bit mode** each byte is sent as two halves, one after the other. It is half the speed - and completely irrelevant, because the display only needs updating a few times a second while the chip runs sixteen million times faster.

SESSION 10

one class

Wiring and Driving the Display

Today you will

- Wire the LCD in 4-bit mode with the contrast pot.
- Get your group name on the screen.
- Learn setCursor and how the two rows are addressed.

LCD weather system - connections

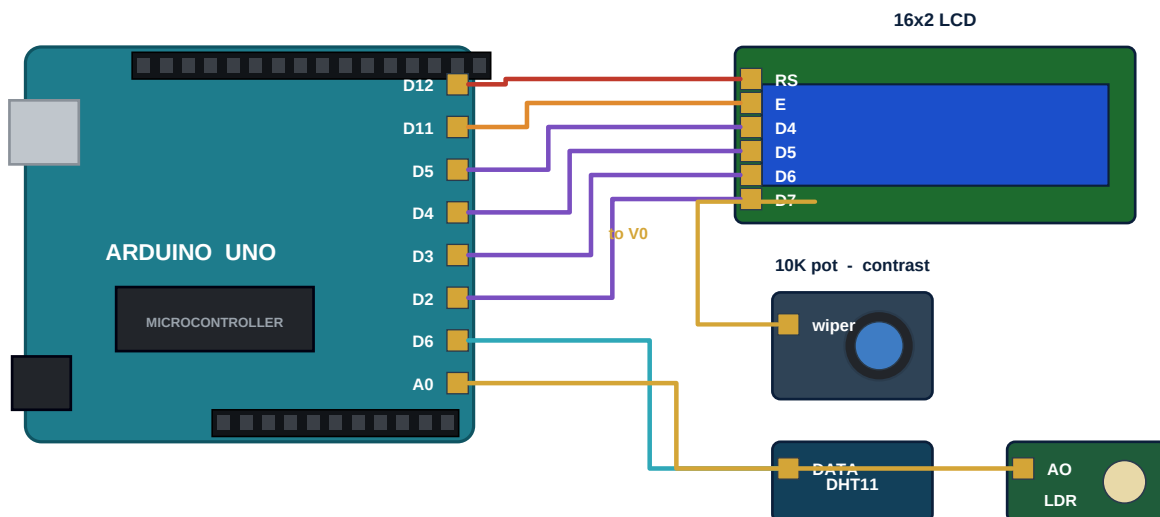


Figure 4.2 - Six control wires run the display in 4-bit mode

LCD pin	Goes to	Note
VSS	GND	
VDD	5V	
V0	Potentiometer wiper	Turn it until the boxes just appear, then back slightly
RS	D12	Command or character
RW	GND	Always write
E	D11	The 'take it now' pulse
D4, D5, D6, D7	D5, D4, D3, D2	Note the deliberate reversal - follow it exactly
A	5V through 220 ohm	Backlight positive
K	GND	Backlight negative

LcdHello.ino

```
#include <LiquidCrystal.h>

// the numbers, in order: RS, E, D4, D5, D6, D7
```



```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
  lcd.begin(16, 2);           // 16 columns, 2 rows
  lcd.setCursor(0, 0);        // column 0, row 0
  lcd.print("PIXIU TECH");
  lcd.setCursor(0, 1);        // column 0, row 1
  lcd.print("Class 9 Lab");
}

void loop() {
}
```

Watch Out

If you see a row of solid white blocks and nothing else, your wiring is probably fine and your **contrast** is wrong - turn the potentiometer slowly. If you see nothing at all, not even blocks, check the backlight and VDD first. Learning to tell those two symptoms apart will save you an hour.

One detail about `setCursor(column, row)` catches everybody: it takes **column first, then row**, and both count from zero. So the last character of the bottom line is `setCursor(15, 1)`. Get the order backwards and your text vanishes off the edge of a screen that only has two rows.

SESSION 11

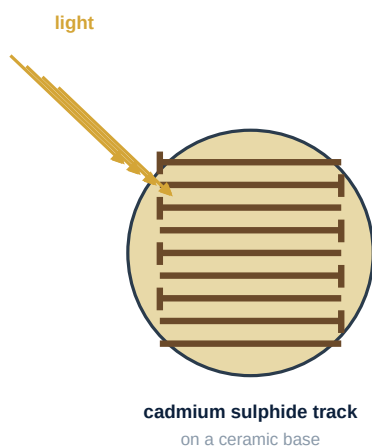
one class

Adding the Sensors and Rotating Screens

Today you will

- Feed the DHT11 and the LDR onto the display.
- Rotate between two screens without freezing the program.
- Track the highest and lowest temperature of the day.

Your third reading comes from the LDR, and it is worth a minute to see what is inside it. Under the clear window sits a zig-zag track of **cadmium sulphide** printed on a ceramic base. The zig-zag is not decoration: it makes the track long and narrow, which magnifies the effect you are about to measure.



What the light actually does

In the dark

electrons stay locked in place
resistance high, about 1,000,000 ohms

In bright light

photons knock electrons free
resistance low, a few hundred ohms

This is called photoconductivity. Notice what the sensor does not do: it never measures brightness in any unit. It only changes its resistance, and the module turns that into a voltage your Arduino can read.

Figure - Inside an LDR: light knocks electrons loose and current flows more easily

Behind the Scenes

In darkness the electrons in that material are bound in place, so almost nothing flows and the resistance can reach a million ohms. When light lands, each photon carries just enough energy to knock an electron free, and every freed electron makes the material a little more conductive. In bright sun the resistance can fall to a few hundred ohms - a change of several thousand times. The effect is called **photoconductivity**, and the same principle runs the light sensor that dims your phone screen.

Notice what the LDR does **not** do. It never reports brightness in any unit. It only changes its resistance, and the module turns that into a voltage that your ADC can read. What that number means is entirely for you to decide - which is exactly the job of a threshold.

Three readings will not fit comfortably on thirty-two characters, so your display must take turns. The obvious way is a long `delay()` - and it is the wrong way, because during a delay the Arduino is completely frozen and cannot read a sensor or respond to anything.

The professional approach uses `millis()`, which reports how many milliseconds the board has been running. Instead of saying *stop here for three seconds*, you say *if three seconds have passed since last time, switch the screen*. The program never stops, and it can still do other work in between.

LcdWeatherSystem.ino

```
#include <LiquidCrystal.h>
#include <DHT.h>

LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
#define DHTPIN 6
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

int ldrPin = A0;

float tMin = 99.0, tMax = -99.0; // start deliberately out of range

unsigned long lastSwap = 0;
int screen = 0;

void setup() {
  lcd.begin(16, 2);
  dht.begin();
  Serial.begin(9600);
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  int light = analogRead(ldrPin);

  if (!isnan(t)) { // only record a real reading
    if (t < tMin) { tMin = t; }
    if (t > tMax) { tMax = t; }
  }

  if (millis() - lastSwap > 3000) { // three seconds have passed
    lastSwap = millis();
    screen = 1 - screen; // flip between 0 and 1
    lcd.clear();
  }

  if (screen == 0) {
    lcd.setCursor(0, 0);
    lcd.print("Temp: "); lcd.print(t); lcd.print(" C");
    lcd.setCursor(0, 1);
    lcd.print("Humid: "); lcd.print(h); lcd.print(" %");
  }
}
```

```

    } else {
      lcd.setCursor(0, 0);
      lcd.print("Light: "); lcd.print(light);
      lcd.setCursor(0, 1);
      lcd.print("L:"); lcd.print(tMin);
      lcd.print(" H:"); lcd.print(tMax);
    }

    delay(500);
  }

```

Behind the Scenes

Look at `screen = 1 - screen;`. If `screen` is 0 it becomes 1; if it is 1 it becomes 0. One line of arithmetic replaces an entire if-else. Small tricks like this are everywhere in professional code - though remember that clever code is only good code when it is still readable.

11.1 Making Your Own Character

The HD44780 stores 208 ready-made characters, but it also keeps eight slots empty for you. A character is a 5 by 8 grid, so you can describe one as eight rows of five bits. Here is a degree symbol and a small thermometer.

CustomCharacter.ino

```

byte degree[8] = {
  0b00110,
  0b01001,
  0b01001,
  0b00110,
  0b00000,
  0b00000,
  0b00000,
  0b00000
};

void setup() {
  lcd.begin(16, 2);
  lcd.createChar(0, degree); // store it in slot 0
  lcd.setCursor(0, 0);
  lcd.print("Temp 28");
  lcd.write(byte(0));        // draw your own character
  lcd.print("C");
}

```

Read the ones and zeros as a picture: each 1 is a dot that lights, each 0 stays dark. Draw your design on squared paper first, then write down each row as five binary digits. The 0b at the front simply tells the compiler that what follows is binary rather than an ordinary number.

Challenge

Add a comfort message on the second row: DRY below 40 percent humidity, OK between 40 and 65, HUMID above 65. Then add a third screen showing how long the system has been running, using `millis()` / 60000 to get minutes.

New word	What it means
Liquid crystal	A state of matter that flows like a liquid but aligns like a crystal.
Backlight	The LED behind an LCD that supplies all the light you see.
HD44780	The standard controller chip on the back of a character LCD.
4-bit mode	Sending each byte as two halves, to save Arduino pins.
millis()	How long the board has been running, in milliseconds.
Non-blocking	Code that waits without freezing everything else.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 4**A. Choose the correct answer**

1. An LCD dot produces
(a) its own light (b) no light of its own (c) heat (d) sound
2. The contrast of the display is set by
(a) the backlight (b) the potentiometer on V0 (c) the RS pin (d) the library
3. The HD44780 is
(a) a sensor (b) the display's controller chip (c) a resistor (d) a library
4. 4-bit mode is used in order to
(a) run faster (b) save Arduino pins (c) improve contrast (d) reduce power
5. `setCursor(15, 1)` places the cursor at
(a) row 15, column 1 (b) the last character of the bottom row (c) the first character (d) off screen
6. Rotating screens with `millis()` instead of `delay()` means the program
(a) runs faster (b) never freezes (c) uses less memory (d) needs no library

B. Fill in the blanks

- 1 Every character on the display is drawn on a _____ by _____ grid of dots.
- 2 The RW pin is tied to ground because we only ever _____.
- 3 Solid white blocks on the screen usually mean the _____ is wrong.
- 4 `millis()` reports how long the board has been _____.

C. Answer in one or two lines

- 1 Explain how applying a voltage makes an LCD dot go dark.
- 2 Why does an LCD need a backlight when a phone screen does not?
- 3 Explain what `screen = 1 - screen;` does and why it works.
- 4 Why is a long `delay()` a poor way to rotate between screens?

UNIT 5

Final Project: Line Following Robot

Sessions 12, 13, 14 and 15

Everything so far has stayed on the desk. This one drives itself. Two sensors look at the floor, a driver board handles the power, and a loop that runs many times a second keeps the machine on a painted path.

By the end of this unit you will be able to:

- Explain how an IR sensor distinguishes black from white.
- Explain how an H-bridge reverses a motor.
- Build a closed feedback loop and tune it.
- Debug a system where the machine's own movement changes what it senses.

SESSION 12

one class

Inside an IR Sensor, and Inside a Motor Driver

Today you will

- Learn how an IR pair reads a surface.
- Understand what an H-bridge does.
- Agree the team roles and the battery rules.

An IR sensor module carries two tiny components side by side. One is an **infrared LED** that shines light we cannot see. The other is a **phototransistor** that conducts more when infrared falls on it. Both point at the floor, a few millimetres away.

A white surface scatters that infrared straight back, the phototransistor sees plenty, and the module's comparator outputs LOW. A black surface absorbs almost all of it, the phototransistor sees nothing, and the output goes HIGH. That is the whole mechanism - and it is why the sensor works just as well in a dark room as in a bright one, because it brings its own light.

How an IR sensor tells black from white

It shines invisible infrared light down and watches how much bounces back

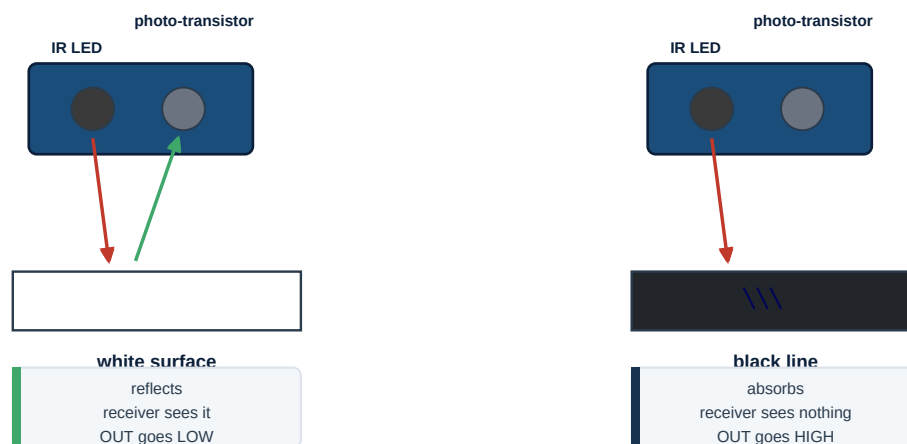


Figure 5.1 - Two tiny components, one very useful trick

Behind the Scenes

The blue trimmer on the module sets the level at which the comparator flips. Too sensitive and every shadow reads as black; too dull and the line disappears. Mounting height matters just as much: about 5 to 10 millimetres above the floor is right. Higher and the reflected light spreads too thin; lower and the sensor sees only its own glow.

12.1 Why a Motor Cannot Touch a Pin

An Arduino output pin can supply about 20 milliamps safely. A BO motor starting under load can demand several hundred - more than ten times as much. Connect one directly and the pin does not merely fail, it burns out permanently.

The **L298N** solves this. Inside are two **H-bridges**, each a set of four electronic switches arranged around a motor in the shape of the letter H. Close one diagonal pair and current flows one way; close the other pair and it flows the other way. The Arduino only ever sends tiny signals telling those switches what to do - it never touches the motor current at all.

The L298N: two switches decide which way a motor turns

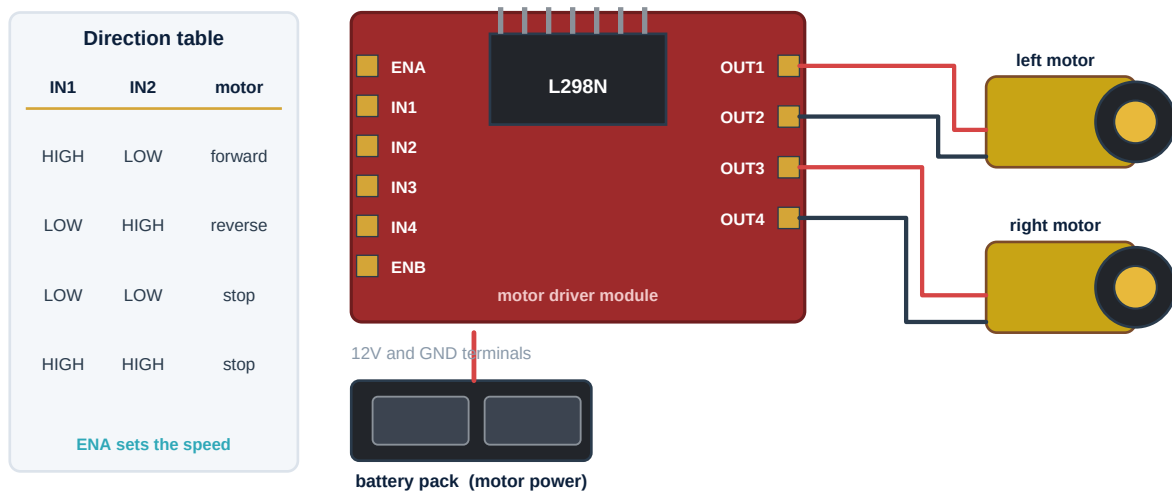


Figure 5.1 - IN pins choose the direction, EN pins choose the speed

12.2 Why the Motor Has a Gearbox

Look closely at your BO motor. The small silver cylinder at the back is the actual motor, and it spins astonishingly fast - several thousand turns a minute - but with almost no strength. Bolted to the front is a **gearbox**, a stack of plastic gears that trades that speed for force.

The bargain is exact. Gear the output down by a factor of fifty and you turn fifty times slower, but with roughly fifty times the turning force. That is why your wheel spins at a sensible 100 revolutions per minute instead of uselessly fast, and why it can actually push a chassis, a battery and a board along the floor.

Quantity	Bare motor	After the gearbox	Why it matters
Speed	very high	about 100 rpm	a fast wheel is uncontrollable
Torque, the turning force	tiny	much larger	it must move the robot's weight
Current at start-up	high	high	this is why a driver board is needed

Watch Out

Two things stop more school robots than any code error. First, the small jumper caps over ENA and ENB: while fitted, the motors run flat out and your speed control does nothing. Remove them. Second, the wire from **Arduino GND to L298N GND**: without it, the two boards do not agree on what zero volts means, and nothing works even though everything looks correct.

Role	Responsible for	Name
Chassis builder	Mounting motors, wheels, caster, battery and sensor height	_____
Wiring engineer	Every connection, checked twice, and the shared ground	_____
Programmer	Typing, uploading and tuning speed and thresholds	_____
Recorder	The notebook: test runs, faults and what fixed them	_____

SESSION 13

one class

Building the Drive Base

Today you will

- Assemble the chassis, motors, wheels and battery.
- Mount the IR sensors at the right height and spacing.
- Test all four motions before adding any sensor logic.

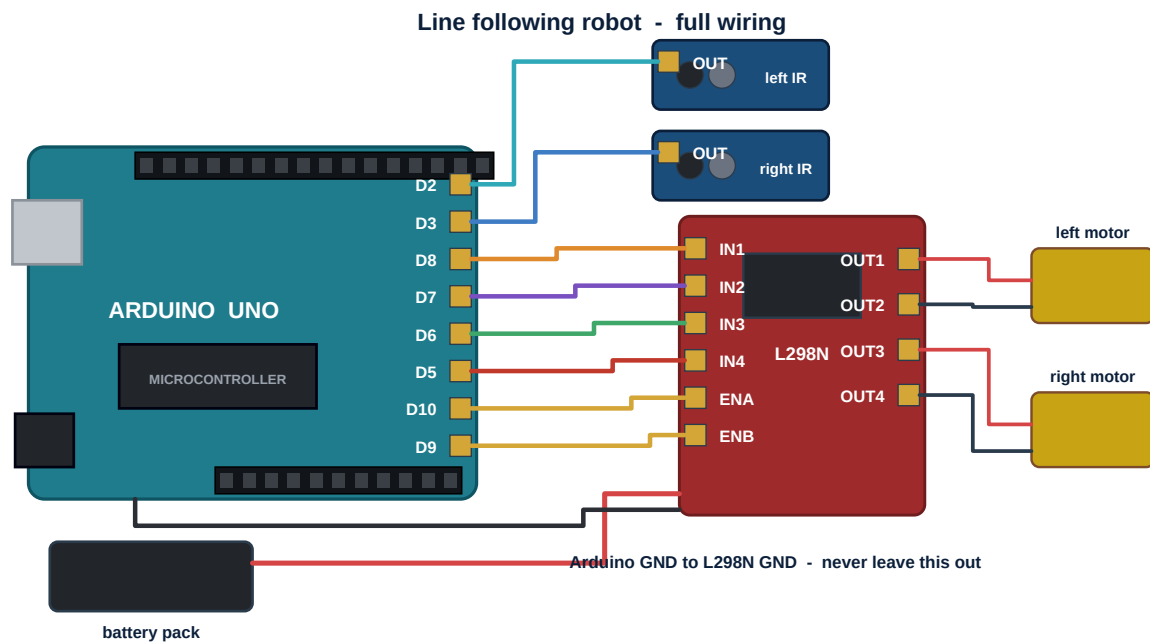


Figure 5.3 - Two eyes, one driver, two wheels

Connection	Goes to	Why
Left IR OUT	D2	Left eye
Right IR OUT	D3	Right eye
L298N IN1 / IN2	D8 / D7	Left motor direction
L298N IN3 / IN4	D6 / D5	Right motor direction
L298N ENA / ENB	D10 / D9	Speed for each motor
L298N OUT1-4	Motor terminals	Power to the wheels
L298N 12V / GND	Battery + / Battery -	Motor power
L298N GND	Arduino GND	Shared ground. Never leave this out.

- Mount the two IR sensors about **2 to 3 centimetres apart**, straddling the line, both pointing straight down.
- Keep them **5 to 10 millimetres** above the floor, and at the very front of the chassis.
- Mount both motors so the wheels are parallel; a few degrees out and the robot curves.
- Keep the battery low and central, and route wires clear of the wheels.

DriveBaseTest.ino

```
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 130;           // start slow. 0 to 255.

void setup() {
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
}
```



```
void forward() {
  analogWrite(ena, speed);  analogWrite(enb, speed);
  digitalWrite(in1, HIGH);  digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH);  digitalWrite(in4, LOW);
}

void left() {                // left wheel slows, right keeps going
  analogWrite(ena, 0);       analogWrite(enb, speed);
  digitalWrite(in3, HIGH);   digitalWrite(in4, LOW);
}

void right() {
  analogWrite(ena, speed);   analogWrite(enb, 0);
  digitalWrite(in1, HIGH);   digitalWrite(in2, LOW);
}

void stopMotors() {
  analogWrite(ena, 0);  analogWrite(enb, 0);
}

void loop() {           // test all four before going further
  forward();  delay(1200);
  left();    delay(600);
  right();   delay(600);
  stopMotors(); delay(1500);
}
```

Watch Out

If a wheel spins the wrong way, do not rewrite the code - swap that motor's two wires at the L298N terminals. It is faster, and it is what a professional would do.

SESSION 14

one class

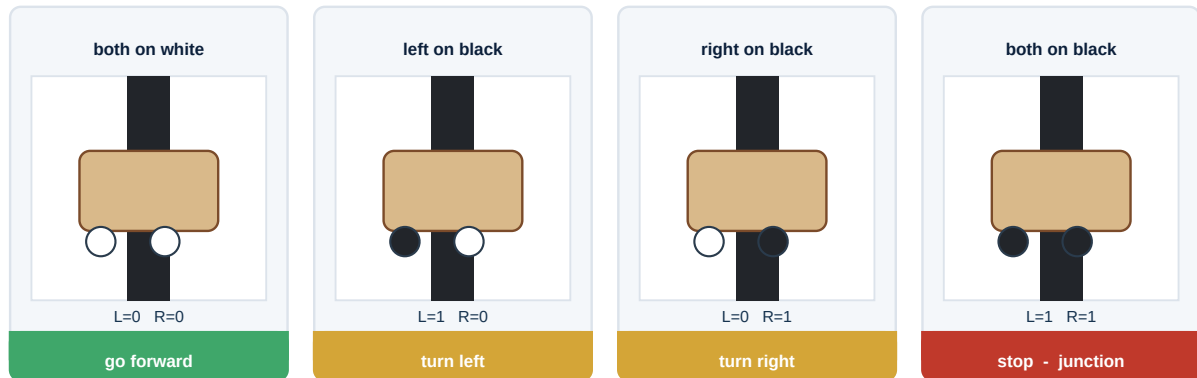
The Four-Case Algorithm

Today you will

- Work out every combination the two sensors can produce.
- Write the line-following program.
- See a closed feedback loop working for the first time.

Two sensors, each with two possible states, gives exactly four combinations - and that is the entire algorithm. Write them down before you write any code.

Four cases, and that is the entire line-following algorithm



A filled circle means that sensor is over black. HIGH means black, LOW means white.

Figure 5.2 - A closed feedback loop: the robot's own moves change what it sees next

What makes this genuinely different from everything before is that it is a **closed feedback loop**. The robot's own movement changes what the sensors see next, which changes the next movement. Nothing else in this book behaves that way, and it is why tuning matters so much: too fast and the robot overshoots before it can correct, too slow and it crawls.

LineFollowingRobot.ino

```
int leftIR = 2, rightIR = 3;
int in1 = 8, in2 = 7, in3 = 6, in4 = 5;
int ena = 10, enb = 9;
int speed = 130;

void setup() {
  pinMode(leftIR, INPUT);
  pinMode(rightIR, INPUT);
  pinMode(in1, OUTPUT); pinMode(in2, OUTPUT);
  pinMode(in3, OUTPUT); pinMode(in4, OUTPUT);
  pinMode(ena, OUTPUT); pinMode(enb, OUTPUT);
  Serial.begin(9600);
  delay(2000);          // time to place the robot and step back
}

void forward() {
  analogWrite(ena, speed); analogWrite(enb, speed);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void turnLeft() {
  analogWrite(ena, 0);      analogWrite(enb, speed);
  digitalWrite(in3, HIGH); digitalWrite(in4, LOW);
}

void turnRight() {
  analogWrite(ena, speed); analogWrite(enb, 0);
  digitalWrite(in1, HIGH); digitalWrite(in2, LOW);
}

void stopMotors() {
  analogWrite(ena, 0); analogWrite(enb, 0);
}

void loop() {
```

```

int L = digitalRead(leftIR);    // HIGH means black
int R = digitalRead(rightIR);

Serial.print(L); Serial.print(" "); Serial.println(R);

if (L == LOW && R == LOW) {
    forward();                  // both on white - on track
} else if (L == HIGH && R == LOW) {
    turnLeft();                 // drifting right - steer back
} else if (L == LOW && R == HIGH) {
    turnRight();                // drifting left - steer back
} else {
    stopMotors();               // both black - a junction or the end
}

delay(10);                      // a short loop keeps it responsive
}

```

Watch Out

Some IR modules output HIGH on black and others output LOW. Before blaming your algorithm, print L and R and slide a piece of black tape under each sensor by hand. If yours are reversed, swap HIGH and LOW in the four conditions - do not rewire anything.

SESSION 15

one class

Tuning, Track Testing and Your Demonstration**Today you will**

- Tune speed and sensor height on the real track.
- Pass every acceptance test.
- Present your robot in three minutes.

What you see	Most likely cause	What to do
Nothing happens at all	Missing Arduino-to-L298N ground wire	Add it. This is the number one fault.
Motors run flat out, speed ignored	ENA and ENB jumper caps still fitted	Remove both caps
Robot wobbles violently along the line	Speed too high for the correction rate	Lower speed to about 110 and retest
Robot leaves the line on curves	Sensors too close together, or too far apart	Adjust spacing to straddle the line snugly
Reacts to shadows	Trimmer set too sensitive	Turn the blue screw slightly and retest
Stops randomly on a straight	Both sensors briefly reading black	Check sensor height and tape width

Acceptance test	Result	Tick
Follows a 4 metre track with two curves, no rescue	_____	■
Recovers when placed slightly off the line	_____	■
Stops at a junction where both sensors see black	_____	■
Three consecutive runs with no failure	_____	■
Runs for 3 minutes without resetting	_____	■

15.1 Beyond Four Cases: Steering Proportionally

Your robot steers with only three answers: straight, hard left, hard right. That is why it wobbles. A real machine does something smarter - it steers **in proportion** to how far off course it is. A small drift gets a small correction; a large drift gets a large one.

With two digital sensors you cannot measure *how far* off you are, only *whether* you are. But the modules also have an AO pin. Read those on A0 and A1, subtract one from the other, and you get an **error** value: near zero when centred, large and positive when drifting one way, large and negative the other. Then set each motor's speed from that error.

Proportional steering - the idea in eight lines

```
int error = analogRead(A0) - analogRead(A1); // how far off centre, and which way
int correction = error / 6; // 6 is your tuning number

int leftSpeed = speed - correction;
int rightSpeed = speed + correction;

leftSpeed = constrain(leftSpeed, 0, 255); // never go outside 0 to 255
rightSpeed = constrain(rightSpeed, 0, 255);

analogWrite(ena, leftSpeed);
analogWrite(enb, rightSpeed);
```

This is the first half of something called a **PID controller**, which is used in everything from aircraft autopilots to the temperature control of an industrial oven. You have just met the P. It is worth trying even if it takes a whole session to tune.

15.2 Your Three-Minute Demonstration

- The problem** - one sentence on where line-following robots are actually used.
- How it senses** - hold up an IR module and explain the LED and phototransistor pair.
- How it decides** - show the four-case table and explain one case fully.
- How it acts** - explain the H-bridge in one sentence, then run the robot live.
- What went wrong** - name one fault you found and fixed. Judges value this most.
- What next** - one honest improvement you would make with more time.

Challenge

Add a third IR sensor in the centre. With three sensors you can tell a gentle drift from a sharp turn, and steer proportionally instead of jerking. Then add the flame sensor to the front so the robot stops if it meets a fire - a genuine warehouse safety feature.

New word	What it means
Phototransistor	A component that conducts more when light falls on it.
Comparator	A circuit that flips its output when an input crosses a set level.
H-bridge	Four switches that let a circuit reverse a motor on command.
Shared ground	The wire letting two supplies agree on what zero volts means.
Closed feedback loop	A system whose own output changes its next input.

CHECK WHAT YOU LEARNT

Check Yourself - Unit 5**A. Choose the correct answer**

1. An IR sensor distinguishes black from white by measuring
(a) temperature (b) reflected infrared light (c) sound (d) voltage from the floor
2. A black surface makes the sensor output
(a) LOW (b) HIGH (c) nothing (d) 5 volts
3. The blue trimmer on an IR module adjusts
(a) brightness (b) the comparator threshold (c) the speed (d) the colour
4. An H-bridge lets a circuit
(a) measure distance (b) reverse a motor (c) store charge (d) read a sensor
5. The wire most often forgotten is
(a) IR OUT to D2 (b) Arduino GND to L298N GND (c) the LED wire (d) the USB cable
6. A system whose own output changes its next input is called
(a) an open loop (b) a closed feedback loop (c) a latch (d) a comparator

B. Fill in the blanks

1. An IR module contains an infrared _____ and a _____.
2. IR sensors should sit about _____ to _____ millimetres above the floor.
3. Both sensors reading black usually means a _____.
4. If the robot wobbles violently, the _____ is probably too high.

C. Answer in one or two lines

1. Explain why an IR sensor works equally well in a dark room and a bright one.
2. Write out all four sensor combinations and the action each one requires.
3. Explain what a closed feedback loop is, using your robot as the example.
4. Which of the three projects this year taught you the most, and why?

Extra Challenges

Finished early? Pick any of these. They all use parts you already have.

#	Challenge	Uses
1	A two-stage fire alarm: warning first, full alarm on confirmation	flame + DHT11
2	An alarm that shows how many times it has triggered today	LCD + counter
3	A night-mode alarm that flashes silently after dark	LDR + buzzer
4	A scrolling message on the LCD, one character at a time	LCD + millis()
5	A custom character of your own design on the display	createChar()
6	A comfort index combining temperature and humidity into one number	DHT11 + maths
7	A robot that counts junctions as it follows the line	2 IR + counter
8	A robot that stops for 3 seconds at each junction, then continues	2 IR + millis()
9	A robot with three IR sensors that steers proportionally	3rd IR module
10	A line follower that shows its speed on the LCD as it drives	LCD + robot

MIXED REVISION

Twenty Questions to Test Yourself

- 1 State Ohm's law and explain each symbol.
- 2 Which chip on the Arduino Uno is the actual computer?
- 3 What does the 16 MHz crystal do?
- 4 Where does your compiled program live, and where do variables live?
- 5 Describe what the bootloader does.
- 6 How many different values can a 10-bit ADC report?
- 7 Work out the size of one ADC step across a 5 volt range.
- 8 What does a flame sensor actually detect?
- 9 Why can sunlight fool a flame sensor?
- 10 Explain why two sensors must agree before your alarm sounds.
- 11 What does it mean for an alarm to latch?
- 12 Why does a push switch need a pull-down resistor?
- 13 Explain the difference between a false positive and a false negative.
- 14 How does applying a voltage make an LCD dot go dark?
- 15 Why is 4-bit mode used to drive the display?
- 16 What does `setCursor(0, 1)` do?
- 17 Why is `millis()` better than `delay()` for rotating screens?
- 18 How does an IR sensor tell black from white?
- 19 Why must the Arduino and the L298N share a ground?
- 20 Name one improvement you would make to your robot with one more session.

Wiring Reference - All Three Projects

Keep this page open while you build. Every pin used this year, in one place.

Project 1 - Fire Security Alarm

Arduino pin	Connects to
D2	Flame sensor DO
A0	Flame sensor AO
D4	DHT11 DATA
D7	Push switch, with 10K to GND
D8	Buzzer +
D9	Orange LED through 220 ohm
D10	Blue LED through 220 ohm
5V and GND	All module VCC and GND

Project 2 - LCD Weather System

Arduino pin	Connects to
D12	LCD RS
D11	LCD E
D5, D4, D3, D2	LCD D4, D5, D6, D7 in that order
D6	DHT11 DATA
A0	LDR module AO
-	LCD V0 to the potentiometer wiper
-	LCD VSS and RW to GND, VDD to 5V
-	LCD A to 5V through 220 ohm, K to GND

Project 3 - Line Following Robot

Arduino pin	Connects to
D2	Left IR sensor OUT
D3	Right IR sensor OUT
D8, D7	L298N IN1, IN2 - left motor direction
D6, D5	L298N IN3, IN4 - right motor direction
D10, D9	L298N ENA, ENB - speed
GND	L298N GND - the shared ground, never omit
-	L298N 12V and GND to the battery pack
-	L298N OUT1-4 to the two motors

Words to Know

Word	Meaning
ADC	The circuit that turns a voltage into a number.
Analog	A value that can be anything within a range.
ATmega328P	The microcontroller chip that is the real computer on the board.
Backlight	The LED behind an LCD that supplies all the light you see.
Boolean	A variable that holds only true or false.
Bootloader	A built-in program that receives your code and writes it to flash.
Closed feedback loop	A system whose own output changes its next input.
Comparator	A circuit that flips its output when an input crosses a set level.
Debounce	Ignoring the brief chatter of a switch as it makes contact.
EEPROM	Small memory that keeps data after the power is removed.
False positive	Alarming when there is nothing to alarm about.
False negative	Staying silent when there really is something.
Flash memory	Where your compiled program is stored.
Floating input	An unconnected pin, reading randomly because of stray noise.
H-bridge	Four switches that let a circuit reverse a motor.
HD44780	The standard controller chip on a character LCD.
Infrared	Light just beyond red, invisible to us.
Latching	Staying on once triggered, until deliberately cleared.
Library	Ready-made code you include and use.
Liquid crystal	A state of matter that flows yet aligns like a crystal.
millis()	How long the board has been running, in milliseconds.
Non-blocking	Code that waits without freezing everything else.
Ohm's law	$V = I \times R$.
Photodiode	A component that produces current when light falls on it.
Phototransistor	A component that conducts more when light falls on it.
Pull-down resistor	A resistor holding an input at 0V until something pulls it high.
Resolution	The smallest change a measuring system can detect.
Shared ground	The wire letting two supplies agree on what zero volts means.
SRAM	Memory holding every variable while the program runs.
System	Several parts working together, where the whole exceeds the parts.

Answer Key

Section A only. Sections B and C are for you and your trainer to discuss - there is often more than one good answer.

Check Yourself - Unit 1

Q	Answer	The correct option
1	(b)	$V = I \times R$
2	(b)	13.6 mA
3	(a)	columns of five
4	(b)	a coded sequence of pulses
5	(b)	hold the input at a known level
6	(b)	has several parts working together

Check Yourself - Unit 2

Q	Answer	The correct option
1	(b)	the ATmega328P chip
2	(b)	the clock
3	(b)	flash memory
4	(c)	1024 values
5	(a)	4.9 millivolts
6	(b)	receive your code and write it to flash

Check Yourself - Unit 3

Q	Answer	The correct option
1	(c)	infrared light
2	(b)	an optical filter
3	(b)	reduce false alarms
4	(b)	a human resets it
5	(b)	floating
6	(b)	false negative

Check Yourself - Unit 4

Q	Answer	The correct option
1	(b)	no light of its own
2	(b)	the potentiometer on V0
3	(b)	the display's controller chip
4	(b)	save Arduino pins
5	(b)	the last character of the bottom row
6	(b)	never freezes

Check Yourself - Unit 5

Q	Answer	The correct option
1	(b)	reflected infrared light
2	(b)	HIGH
3	(b)	the comparator threshold
4	(b)	reverse a motor
5	(b)	Arduino GND to L298N GND
6	(b)	a closed feedback loop

My Project Log

Fill one row after every class. This is your evidence, and it counts towards your assessment.

Session	What I built or learnt	A problem I hit	How I fixed it
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

A last word

This year you stopped treating components as magic boxes. You know what is inside the chip, what an ADC really measures, why a flame sensor is not a thermometer, and why an LCD needs a light behind it. That habit - always asking how the thing actually works - is worth far more than any single project in this book.